

UMESH'S INDIVIDUAL TASK SHEET

Role: Mobile Developer + n8n Automation Specialist (Team B)

Team Partner: Atharav

Timeline: 3 Months (90 Days)

Total Hours: 864 hours (72 hours/week × 12 weeks)

Working Days: Monday to Saturday (12 hours/day)

Rest Day: Sunday (Mandatory - NO CODING!)

YOUR PROFILE & RESPONSIBILITIES

Primary Focus Areas:

1. **Mobile App Development** (50% time)
 - React Native (Expo) mobile apps
 - Student app (test taking, practice)
 - Teacher app (test creation, attendance)
 - Parent app (notifications, progress tracking)
2. **n8n Workflow Automation** (30% time)
 - **YOUR SPECIALTY!**
 - OCR paper generation workflows
 - Notification workflows
 - Automated report generation
 - Data synchronization
3. **Frontend Support** (15% time)
 - Help Atharav with web UI components
 - Shared component library
 - Testing and bug fixes
4. **Integration & Testing** (5% time)
 - API integration in mobile apps
 - E2E testing
 - App store submission

Your Strengths (Leverage These):

- Strong automation mindset
- Good at connecting different systems
- Mobile development skills

- Problem-solving creativity

What You'll Learn:

- Professional React Native development
- **n8n workflow automation (become expert!)**
- Mobile app deployment (Play Store)
- Push notifications and real-time features
- API integration patterns



MONTH 1: FOUNDATION (WEEKS 1-4)

WEEK 1: MOBILE APP SETUP & AUTH (72 hours)

Your Focus: Set up React Native project and build auth screens

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Project Setup

Hour-by-Hour Breakdown:

- **Hours 1-6:** React Native + Expo setup

[] Install Expo CLI:

```
npm install -g expo-cli
```

○

[] Create new Expo project:

```
npx create-expo-app@latest coaching-app-mobilecd coaching-app-mobile
```

○

[] Install dependencies:

```
# Navigationnpx expo install @react-navigation/native @react-navigation/stack
@react-navigation/bottom-tabsnpx expo install react-native-screens
react-native-safe-area-context# Supabasenpm install @supabase/supabase-js# Other
essentialsnpx expo install expo-camera expo-notifications expo-secure-storem
pm install zustand axios react-hook-form
```

○

- **Hours 7-12:** Project structure

[] Set up folder structure:

/src /screens (all app screens) /auth (login, signup) /student (student app screens)
/teacher (teacher app screens) /parent (parent app screens) /components (reusable
components) /navigation (navigation setup) /services (API calls) /store (Zustand state
management) /utils (helper functions) /hooks (custom hooks)

-
- [] Set up navigation (stack + tabs)
- [] Test on iOS Simulator & Android Emulator
- **Hours 13-18:** Supabase client setup

[] Create Supabase client:

```
// services/supabase.tsimport { createClient } from '@supabase/supabase-js';import Constants from 'expo-constants';export const supabase = createClient(Constants.expoConfig?.extra?.supabaseUrl, Constants.expoConfig?.extra?.supabaseAnonKey);
```

-
- [] Store environment variables (app.config.js)
- [] Test connection
- **Hours 19-24:** Basic styling setup
- [] Create theme file (colors, spacing, fonts)
- [] Create reusable style components (Button, Input, Card)
- [] Test on both platforms (iOS/Android)

Deliverables Day 1-2:

-  React Native project set up
-  Navigation working
-  Supabase connected

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Authentication Screens

- **Hours 1-10:** Login screen
 - [] Create LoginScreen.tsx
 - [] Beautiful mobile UI:
 - App logo at top
 - Email input
 - Password input (with eye icon to show/hide)
 - Login button
 - "Forgot password?" link
 - "Don't have account? Sign up" link

- ☐ Form validation (react-hook-form)
- ☐ Loading state (ActivityIndicator)
- **Hours 11-18: Signup screen**
 - ☐ Create SignupScreen.tsx
 - ☐ Multi-step form:
 - Step 1: Choose role (Student/Teacher/Parent) - three cards
 - Step 2: Basic info (name, email, phone)
 - Step 3: Role-specific (class for student, subjects for teacher, etc.)
 - ☐ Progress indicator dots (● ○ ○)
 - ☐ Next/Previous buttons
- **Hours 19-24: Auth integration**

☐ Integrate with Supabase Auth:

```
const handleLogin = async (email: string, password: string) => { const { data, error } = await
supabase.auth.signInWithPassword({ email, password }); if (error) { Alert.alert('Error',
error.message); } else { // Navigate to home screen navigation.navigate('Home'); }}
```

-
- ☐ Store auth state (Zustand)
- ☐ Persist session (expo-secure-store)
- ☐ Auto-login on app open (if session exists)

Deliverables Day 3-4:

- ☒ Login and signup screens working
- ☒ Auth flow integrated

Day 5-6 (Friday-Saturday) - 24 hours

Task: Home Screens & Navigation

- **Hours 1-12: Student home screen**
 - ☐ Create StudentHomeScreen.tsx
 - ☐ Tab navigation:
 - Home (dashboard)
 - Tests (upcoming and past tests)
 - Practice (practice mode)
 - Profile (student profile and settings)
 - ☐ Dashboard cards:
 - Upcoming tests
 - Recent scores
 - Attendance percentage
 - Practice streak
 - ☐ Beautiful card design (shadows, gradients)

- **Hours 13-20:** Teacher home screen
 - ☐ Create TeacherHomeScreen.tsx
 - ☐ Tab navigation:
 - Home (dashboard)
 - Tests (manage tests)
 - Attendance (mark attendance)
 - Profile
 - ☐ Dashboard cards:
 - Classes
 - Pending evaluations
 - Today's schedule
 - Quick actions
- **Hours 21-24:** Parent home screen
 - ☐ Create ParentHomeScreen.tsx
 - ☐ Simple UI (parents need clarity!)
 - ☐ Child selector (if multiple children)
 - ☐ Child's performance summary
 - ☐ Recent notifications
 - ☐ Week 1 wrap-up

Week 1 Complete! 🎉

- Integration Point: Mobile apps connected to backend 

WEEK 2: STUDENT TEST INTERFACE (MOBILE) (72 hours)

Your Focus: Build mobile test interface (simplified from web)

Day 1-4 (Monday-Thursday) - 48 hours

Task: Mobile Test Taking

- **Hours 1-15:** Test list screen
 - ☐ Create TestListScreen.tsx

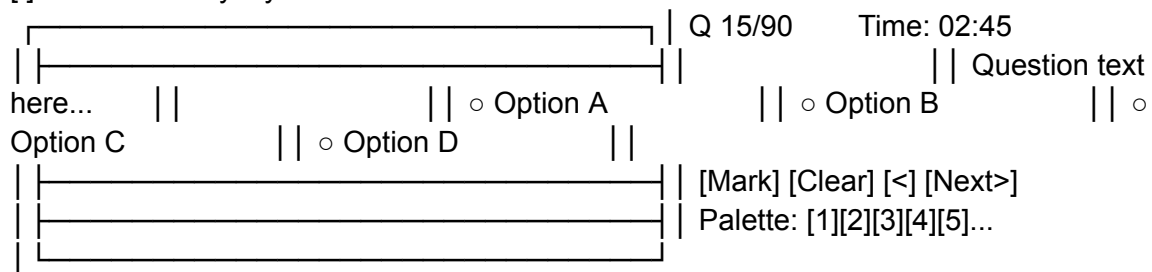
☐ List of assigned tests:

Tomorrow, 10:00 AM ⌚ Duration: 3 hours		 JEE Main Mock Test 1 [Start Test]		⌚	
--	--	--	--	--------------	--

- ☐ Status badges (Scheduled, Live, Completed)
- ☐ Pull-to-refresh
- ☐ Filter (by status)

- **Hours 16-30:** Test start screen
 - ☐ Create TestStartScreen.tsx
 - ☐ Instructions display (scrollable)
 - ☐ Pre-test checks:
 - Good internet connection?
 - Battery > 20%?
 - Quiet environment?
 - ☐ "I'm Ready" button
 - ☐ Request full-screen mode (if supported)
- **Hours 31-45:** Test interface (mobile-optimized)
 - ☐ Create TestInterfaceScreen.tsx

☐ Mobile-friendly layout:



- ☐ Swipe left/right for prev/next question
- ☐ Large touch targets (buttons)
- ☐ Question palette (scrollable horizontally)
- **Hours 46-48:** Answer handling
 - ☐ Radio buttons for MCQ
 - ☐ Number keyboard for numerical
 - ☐ Auto-save answers (every 10 seconds)
 - ☐ Mark for review toggle
 - ☐ Clear response button

Deliverables Day 1-4:

- ☒ Complete mobile test interface
- ☒ Optimized for touch

Day 5-6 (Friday-Saturday) - 24 hours

Task: Timer & Results

- **Hours 1-10:** Timer implementation
 - ☐ Countdown timer (always visible)

- [] Warning notification (5 minutes left)
- [] Auto-submit when time = 0
- [] Store time in AsyncStorage (persist on app close)
- **Hours 11-18: Test submission**
 - [] Submit button
 - [] Summary dialog (answered, unanswered, marked)
 - [] Final submit confirmation
 - [] Loading state
 - [] Success message
- **Hours 19-24: Results screen**
 - [] Create TestResultScreen.tsx
 - [] Score card:
 - Big score display (285/300)
 - Percentage
 - Rank badge (🏆 Rank 2/150)
 - [] Subject-wise breakdown
 - [] "View Answers" button
 - [] Week 2 wrap-up

Week 2 Complete! 🎉

WEEK 3: PRACTICE MODE & NOTIFICATIONS (72 hours)

Your Focus: Build practice mode and push notifications

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Practice Mode

- **Hours 1-15: Practice session screen**
 - [] Create PracticeScreen.tsx
 - [] Single question display
 - [] Submit answer button
 - [] Instant feedback:
 -  Correct (green background)
 -  Wrong (red background, show correct answer)
 - [] Explanation display
 - [] "Next Question" button
- **Hours 16-27: Similar questions flow**
 - [] If wrong, show "Practice Similar" button
 - [] Fetch 5 similar questions from API
 - [] Progress indicator (1/5, 2/5, etc.)
 - [] Celebrate when all 5 correct (confetti animation!)

- **Hours 28-36:** Progress tracking
 - ☐ Create ProgressScreen.tsx
 - ☐ Circular progress bars (subject-wise)
 - ☐ Weak topics list (red tags)
 - ☐ Strong topics list (green tags)
 - ☐ Recommended topics to practice

Deliverables Day 1-3:

- ☒ Practice mode working
 - ☒ Adaptive learning integrated
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Push Notifications

- **Hours 1-15:** Notification setup

☐ Configure Firebase Cloud Messaging:
 npx expo install expo-notifications expo-device

○

☐ Request notification permissions:
 import * as Notifications from 'expo-notifications'; const { status } = await
 Notifications.requestPermissionsAsync();

○

☐ Get FCM token:
 const token = await Notifications.getExpoPushTokenAsync();// Send token to backend

○

- ☐ Store token in backend (link to user)

- **Hours 16-27:** Handle notifications

☐ Foreground notification handler:
 Notifications.setNotificationHandler({ handleNotification: async () => ({ shouldShowAlert:
 true, shouldPlaySound: true, shouldSetBadge: true, }));

○

- ☐ Background notification handler
- ☐ Notification tap handler (navigate to relevant screen)

- **Hours 28-36:** Notification types

- ☐ Test reminder (1 hour before test)
- ☐ Test published (new test available)
- ☐ Results declared (test results out)
- ☐ Attendance marked (parent notification)
- ☐ Low attendance warning
- ☐ Week 3 wrap-up

Week 3 Complete! 🎉

WEEK 4: TEACHER APP FEATURES (72 hours)

Your Focus: Build teacher-specific features on mobile

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Attendance Marking (Mobile)

- **Hours 1-15:** Manual attendance screen
 - ☐ Create AttendanceScreen.tsx
 - ☐ Select class/batch dropdown
 - ☐ Student list with checkboxes
 - ☐ Mark all present button
 - ☐ Submit attendance button
 - ☐ Today's date display
- **Hours 16-27:** Face recognition trigger (mobile)
 - ☐ "Mark via Face Recognition" button
 - ☐ Camera access
 - ☐ Capture photo button
 - ☐ Send to backend for recognition
 - ☐ Show recognized students
 - ☐ Confirm and submit
- **Hours 28-36:** Attendance reports
 - ☐ Create AttendanceReportScreen.tsx
 - ☐ Calendar view (mark present/absent days)
 - ☐ Student attendance percentage
 - ☐ Low attendance alerts

Deliverables Day 1-3:

- ☒ Attendance marking on mobile
 - ☒ Face recognition integrated
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Quick Test Creation (Mobile)

- **Hours 1-18:** Simplified test creation
 - ☐ Create QuickTestScreen.tsx
 - ☐ Simplified form (fewer options than web):
 - Test name
 - Duration
 - Select questions (from favorites or recent)
 - Assign to class
 - ☐ "Create Test" button
 - ☐ Success message
- **Hours 19-30:** Test management
 - ☐ Teacher's test list
 - ☐ View test details
 - ☐ Edit test (limited editing)
 - ☐ Delete test
 - ☐ Publish test
- **Hours 31-36:** Month 1 wrap-up
 - ☐ Test all mobile features
 - ☐ Fix bugs
 - ☐ Month 1 demo

Week 4 Complete! Month 1 Done! 🎉🎉



MONTH 2: N8N AUTOMATION & ADVANCED FEATURES (WEEKS 5-8)

WEEK 5: N8N FUNDAMENTALS (72 hours)

🔥 **YOUR SPECIALTY WEEK - N8N Automation!**

Your Focus: Master n8n and build first workflows

Day 1-2 (Monday-Tuesday) - 24 hours

Task: n8n Deep Learning

- **Hours 1-8:** n8n installation & setup

[] Install n8n:

Docker (recommended) docker run -d --name n8n \ -p 5678:5678 \ -v
~/.n8n:/home/node/.n8n \ n8nio/n8n

-
- [] Access n8n (http://localhost:5678)
- [] Create account
- [] Explore interface
- **Hours 9-16:** Learn n8n basics
 - [] Watch n8n tutorials (YouTube - 3 hours)
 - [] Read n8n documentation (2 hours)
 - [] Understand workflow concepts:
 1. Nodes (trigger, action, condition)
 2. Connections (flow of data)
 3. Expressions (data transformation)
 4. Error handling
 - [] Build 3 practice workflows:
 1. Webhook → HTTP Request → Email
 2. Schedule → Database Query → Slack
 3. Form Submit → Google Sheets → Notification
- **Hours 17-24:** Advanced n8n concepts
 - [] Variables and expressions
 - [] Conditional logic (IF node)
 - [] Loops (Split In Batches node)
 - [] Error handling (Error Trigger node)
 - [] Sub-workflows
 - [] Practice with sample data

Deliverables Day 1-2:

-  n8n installed and understood
 -  Built practice workflows
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: First Real Workflow - Notifications

- **Hours 1-12:** Design notification workflow

[] Workflow design:

Webhook (test published) ↓ HTTP Request (get assigned students) ↓ Split In Batches
(process students one by one) ↓ Firebase Cloud Messaging (send push notification)
↓ Supabase (log notification sent)

-
- **Hours 13-20:** Build workflow
 - ☐ Create workflow in n8n
 - ☐ Set up webhook trigger
 - ☐ Configure HTTP Request node (call backend API)
 - ☐ Configure FCM node (send push notification)
 - ☐ Configure Supabase node (insert log)
 - ☐ Test with sample data
- **Hours 21-24:** Error handling
 - ☐ Add error handling:
 - If student doesn't have FCM token, skip
 - If notification fails, retry once
 - Log errors to database
 - ☐ Test error scenarios

Deliverables Day 3-4:

- ☒ Notification workflow working
 - ☒ Error handling implemented
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Scheduled Workflows

- **Hours 1-12:** Daily report workflow
 - ☐ Workflow: Generate daily attendance report
 - Trigger: Cron (every day at 8 PM)
 - Query: Get today's attendance data
 - Process: Calculate percentages
 - Generate: Create PDF report
 - Send: Email to admin
- **Hours 13-20:** Weekly summary workflow
 - ☐ Workflow: Send weekly progress report to parents
 - Trigger: Cron (every Sunday 9 PM)
 - Query: Get week's test scores, attendance
 - Process: Calculate averages, trends
 - Generate: Create summary
 - Send: Email/SMS to parents
- **Hours 21-24:** Week 5 wrap-up
 - ☐ Test all workflows
 - ☐ Monitor for errors
 - ☐ Document workflows

Week 5 Complete! N8N Expert in the Making! 🎉

WEEK 6: OCR PAPER GENERATION WORKFLOW (72 hours)

🔥 YOUR MOST COMPLEX WORKFLOW!

Your Focus: Build complete OCR to paper workflow

Day 1-3 (Monday-Wednesday) - 36 hours

Task: OCR Workflow Design & Implementation

- **Hours 1-10:** Workflow design (on paper first!)

☐ Draw complete workflow:

1. Webhook Trigger (teacher uploads image) ↓ 2. Download Image (from URL) ↓ 3. Preprocess Image (Python script via HTTP Request) ↓ 4. OCR (Google Vision API or Tesseract) ↓ 5. Text Cleaning (OpenAI API) ↓ 6. Layout Detection (Python script) ↓ 7. Generate Word Doc (Python script) ↓ 8. Upload to Supabase Storage ↓ 9. Notify Teacher (email + push)

- - ☐ Identify required nodes and APIs
 - ☐ Plan error handling at each step
- **Hours 11-20:** Build OCR workflow (Part 1)
 - ☐ Create new workflow in n8n
 - ☐ Webhook trigger (file uploaded)
 - ☐ HTTP Request node (download image from URL)

☐ Google Cloud Vision API node:

```
{ "requests": [{ "image": {"content": "base64_image"}, "features": [{"type": "TEXT_DETECTION"}] } ] }
```

- - ☐ Extract text from response
- **Hours 21-30:** Build OCR workflow (Part 2)

☐ OpenAI API node (text cleaning):

Prompt: "Clean this OCR text. Fix spelling, punctuation. Maintain question numbering. Convert math to LaTeX if needed. Text: {{ \$json['text'] }}"

- - ☐ Python script node (layout detection)
 - ☐ Test workflow up to this point

- **Hours 31-36:** Error handling
 - [] Add error handling:
 - If OCR fails → send error notification
 - If text cleaning takes too long → timeout and use raw text
 - If any step fails → log error and notify teacher
 - [] Add retry logic (max 2 retries)

Deliverables Day 1-3:

-  OCR workflow 70% complete
 -  Text extraction and cleaning working
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Document Generation & Storage

- **Hours 1-15:** Document generation node

[] HTTP Request node (call Python service):

POST /api/paper/generate-documentBody: { "text": "cleaned_text", "format": "docx",
"template": "institute_template" }

-
- [] Receive generated document (base64 or URL)
- **Hours 16-27:** Storage and delivery

[] Supabase node (upload document):

// Upload to Supabase Storage bucket: 'generated-papers', path:
`papers/\${teacherId}/\${timestamp}.docx`

-
- [] Get public URL

[] Supabase node (save record in database):

INSERT INTO generated_papers (teacher_id, original_file_url, generated_file_url, status)
VALUES (...)

-
- **Hours 28-36:** Notification and testing

[] Email node (send completion email):

Subject: Your paper is ready!Body: Click here to download: {{\$json["url"]}}

-

- ☐ Push notification node
- ☐ Test complete workflow end-to-end:
 1. Upload test image
 2. Wait for processing
 3. Receive notification
 4. Download generated paper
 5. Verify quality
- ☐ Week 6 wrap-up

Week 6 Complete! OCR Workflow Working! 🎉📄

WEEK 7: MOBILE PROCTORING & OFFLINE SUPPORT (72 hours)

Your Focus: Build mobile proctoring and offline features

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Mobile Proctoring

- **Hours 1-15:** Tab detection (mobile)

☐ App state detection:

```
import { AppState } from 'react-native'; AppState.addListener('change', (state) => { if
(state === 'background') { // User switched app logViolation('APP_SWITCH'); }});
```

-
- ☐ Show warning dialog
- ☐ Track violation count
- ☐ Auto-submit after 3 violations
- **Hours 16-27:** Webcam monitoring (optional)
 - ☐ Camera access (expo-camera)
 - ☐ Capture snapshots (every 2 minutes)
 - ☐ Upload to backend
 - ☐ Show small preview (corner of screen)
- **Hours 28-36:** Battery and network checks
 - ☐ Battery level check (warn if < 20%)
 - ☐ Network status monitoring (warn if slow/disconnected)
 - ☐ Auto-save answers frequently (prevent data loss)

Deliverables Day 1-3:

- ☒ Mobile proctoring features working
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Offline Support

- **Hours 1-18:** Offline data storage

[] Install AsyncStorage:

```
npx expo install @react-native-async-storage/async-storage
```

○

[] Cache downloaded questions locally:

```
await AsyncStorage.setItem('test_123_questions', JSON.stringify(questions));
```

○

[] Queue answers for upload when online:

```
await AsyncStorage.setItem('pending_answers', JSON.stringify(answers));
```

○

- **Hours 19-30:** Sync logic

[] Detect network status:

```
import NetInfo from '@react-native-community/netinfo';NetInfo.addEventListener(state => {  
if (state.isConnected) { // Online → sync pending data   syncPendingAnswers(); } });
```

○

○ [] Upload queued answers

○ [] Show sync status to user

- **Hours 31-36:** Week 7 wrap-up

○ [] Test offline mode thoroughly

○ [] Test sync when back online

○ [] Demo to team

Week 7 Complete! 🎉

WEEK 8: PARENT APP & N8N WORKFLOWS (72 hours)

Your Focus: Build parent app and more workflows

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Parent App

- **Hours 1-15:** Parent dashboard
 - ☐ Create ParentDashboardScreen.tsx
 - ☐ Child selector (if multiple children)
 - ☐ Overview cards:
 - Today's attendance
 - This week's tests
 - Recent scores
 - Attendance percentage
 - ☐ Beautiful, simple UI (parents need clarity!)
- **Hours 16-27:** Detailed views
 - ☐ Performance screen (charts)
 - ☐ Attendance history (calendar view)
 - ☐ Test results list
 - ☐ Notifications list
- **Hours 28-36:** Parent-teacher communication
 - ☐ Message teacher button
 - ☐ Simple chat interface
 - ☐ Push notifications for new messages

Deliverables Day 1-3:

- ☒ Parent app complete
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: More n8n Workflows

- **Hours 1-15:** Data backup workflow
 - ☐ Scheduled workflow (every day at 2 AM)
 - ☐ Export database to CSV/JSON
 - ☐ Upload to Google Drive or Dropbox
 - ☐ Notify admin (email)
- **Hours 16-27:** Payment reminder workflow
 - ☐ Trigger: 3 days before subscription expires
 - ☐ Query: Get institutes expiring soon
 - ☐ Send: Reminder email + SMS
 - ☐ Log: Reminder sent in database
- **Hours 28-36:** Month 2 wrap-up
 - ☐ Test all n8n workflows
 - ☐ Optimize workflow performance
 - ☐ Document all workflows (write README)

- [] Month 2 demo

Week 8 Complete! Month 2 Done! 🎉🎉



MONTH 3: POLISH & LAUNCH (WEEKS 9-12)

WEEK 9: UI POLISH & ADVANCED WORKFLOWS (72 hours)

Your Focus: Polish mobile apps and build advanced workflows

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Mobile UI Polish

- **Hours 1-18:** Improve animations
 - [] Add smooth transitions between screens
 - [] Loading animations (Lottie)
 - [] Success/error animations
 - [] Gesture animations (swipe, pull-to-refresh)
- **Hours 19-30:** Improve UX
 - [] Better error messages
 - [] Empty states (no tests, no results)
 - [] Skeleton loaders (instead of blank screens)
 - [] Haptic feedback (on button press)
- **Hours 31-36:** Accessibility
 - [] Font scaling (support system font size)
 - [] Color contrast (readable text)
 - [] Screen reader support (basic)

Deliverables Day 1-3:

-  Mobile apps polished
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Advanced n8n Workflows

- **Hours 1-15:** Smart notification workflow
 - [] Intelligent notification timing:
 - Don't send at night (9 PM - 8 AM)

- Batch notifications (max 3 per hour)
 - Priority-based (urgent vs normal)
- [] Implement using n8n Switch and Schedule nodes
- **Hours 19-30:** Analytics report workflow
 - [] Weekly analytics report generation:
 - Query database for week's data
 - Calculate metrics (average score, attendance, etc.)
 - Generate charts (using API or Python)
 - Create PDF report
 - Email to admin/teachers
- **Hours 31-36:** Week 9 wrap-up
 - [] Test all workflows
 - [] Monitor for errors
 - [] Optimize performance

Week 9 Complete! 🎉

WEEK 10: PAYMENT & APP STORE PREP (72 hours)

Your Focus: Integrate payments and prepare for app stores

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Payment Integration (Mobile)

- **Hours 1-18:** Razorpay integration

[] Install Razorpay SDK:

```
npm install react-native-razorpay
```

○

[] Payment checkout flow:

```
import RazorpayCheckout from 'react-native-razorpay';
const options = {
  key: RAZORPAY_KEY,
  amount: amount * 100,
  currency: 'INR',
  name: 'CoachingPro',
  description: 'Subscription Payment',
};
RazorpayCheckout.open(options).then((data) => {
  // Payment success
  verifyPayment(data.razorpay_payment_id);
}).catch((error) => {
  // Payment failed
});
```

○

- **Hours 19-30:** Payment screens
 - [] Pricing screen (show plans)
 - [] Payment method selection

- ☐ Payment success screen
- ☐ Payment failure screen (with retry button)
- **Hours 31-36:** Test payments
 - ☐ Test with Razorpay test keys
 - ☐ Test all payment methods
 - ☐ Test failure scenarios

Deliverables Day 1-3:

- ☒ Payment integration working
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: App Store Preparation

- **Hours 1-15:** App assets
 - ☐ App icon (1024x1024 for iOS, various sizes for Android)
 - ☐ Splash screen
 - ☐ Screenshots (for app stores):
 - Different screen sizes (phone, tablet)
 - At least 5 screenshots showing key features
 - ☐ App description (for stores)
- **Hours 16-27:** Build configuration

☐ Update app.json:

```
{ "expo": { "name": "CoachingPro", "slug": "coaching-pro", "version": "1.0.0", "icon":
"./assets/icon.png", "android": { "package": "com.coachingpro.app", "versionCode": 1
}, "ios": { "bundleIdentifier": "com.coachingpro.app", "buildNumber": "1.0.0" } }}
```

- ☐
- ☐ Set up EAS Build (Expo Application Services)
- **Hours 28-36:** Test build

☐ Create development build:

```
eas build --profile development --platform android
```

- ☐
- ☐ Test on physical device
- ☐ Fix any build errors
- ☐ Week 10 wrap-up

Week 10 Complete! 🎉

WEEK 11: TESTING & BUG FIXES (72 hours)

Your Focus: Thorough testing and bug fixing

Day 1-6 (Monday-Saturday) - 72 hours

Task: Comprehensive Testing

- **Hours 1-20:** Device testing
 - ☐ Test on multiple Android devices:
 - Samsung Galaxy (various models)
 - Pixel phones
 - OnePlus
 - Budget phones (important!)
 - ☐ Test on different screen sizes
 - ☐ Test on different Android versions (8+)
- **Hours 21-40:** Feature testing
 - ☐ Test every feature:
 - Login/Signup
 - Test taking (complete flow)
 - Practice mode
 - Notifications
 - Attendance marking
 - Payment
 - ☐ Test edge cases:
 - Poor network
 - Low battery
 - App in background
 - Device rotation
- **Hours 41-60:** Bug fixing
 - ☐ Create bug spreadsheet
 - ☐ Prioritize bugs (critical, high, medium, low)
 - ☐ Fix critical bugs first
 - ☐ Retest after fixes
- **Hours 61-72:** n8n workflow testing
 - ☐ Test all workflows with real data
 - ☐ Monitor for errors
 - ☐ Fix workflow bugs
 - ☐ Optimize slow workflows

Week 11 Complete! 🎉

WEEK 12: LAUNCH! (72 hours)

Your Focus: Final touches and launch

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Production Build & Submission

- **Hours 1-10:** Production build

[] Create production build:

gradle build --profile production --platform android

-
- [] Test production build thoroughly
- [] Ensure all features work
- [] Performance check (app size, speed)
- **Hours 11-20:** Google Play Store submission
 - [] Create Google Play Console account
 - [] Upload APK/AAB
 - [] Fill in app details (description, screenshots, etc.)
 - [] Set up pricing (free)
 - [] Submit for internal testing track first
- **Hours 21-24:** iOS build (if time permits)
 - [] Create iOS production build
 - [] Test on iOS device
 - [] (Full App Store submission can be done later)

Deliverables Day 1-2:

-  Android app on Play Store (internal testing)
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Final n8n Setup

- **Hours 1-12:** Production n8n
 - [] Move n8n to production server (DigitalOcean droplet)
 - [] Configure production workflows
 - [] Set up monitoring (uptime, errors)
 - [] Test all workflows in production
- **Hours 13-24:** Documentation
 - [] Write n8n workflow documentation:

- What each workflow does
- How to modify
- Troubleshooting guide
- ☐ Create workflow diagrams
- ☐ Video tutorials (screen recordings)

Deliverables Day 3-4:

- ☒ Production n8n running
 - ☒ Complete documentation
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Beta Testing & Launch

- **Hours 1-12:** Beta testing support
 - ☐ Help beta institutes with mobile app
 - ☐ Collect feedback
 - ☐ Fix urgent bugs
- **Hours 13-20:** Monitoring
 - ☐ Monitor app performance (Sentry, Firebase Crashlytics)
 - ☐ Monitor n8n workflows (check logs)
 - ☐ Respond to issues quickly
- **Hours 21-24:** Celebration! 🎉
 - ☐ WE LAUNCHED!
 - ☐ Team celebration
 - ☐ Reflect on learnings
 - ☐ Plan next features

Week 12 Complete! WE LAUNCHED! 🚀🎉

YOUR KEY METRICS

Mobile Development:

- ☐ Screens completed: ____ / ____
- ☐ Features implemented: ____
- ☐ Bugs fixed: ____
- ☐ App size: ____ MB

n8n Workflows:

- [] Workflows built: ____
- [] Workflow success rate: ____%
- [] Average execution time: ____ seconds

Quality:

- [] Crash-free rate: ____%
 - [] App rating (beta testers): ____ / 5
-

YOUR TOOLS

Daily:

- VS Code + React Native Debugger
- Expo Go app (testing)
- Android Studio (emulator)
- n8n interface (<http://localhost:5678>)

Testing:

- Physical Android device
- Postman (API testing)
- Firebase Console (notifications, crashlytics)

n8n:

- n8n web interface
 - n8n docs (docs.n8n.io)
 - n8n community forum
-

PERSONAL GOALS

Technical:

- Master React Native
- **Become n8n expert (unique skill!)**
- Mobile app deployment
- Real-time features (push notifications, WebSockets)

Career:

- Mobile app in Play Store (portfolio!)
- Automation expertise
- Full-stack mobile + backend

- Real product launch
-

FINAL CHECKLIST

By Week 4:

- ☐ Mobile apps functional
- ☐ Auth and test interface working

By Week 8:

- ☐ **N8N WORKFLOWS WORKING!** ★
- ☐ OCR paper generation automated

By Week 12:

- ☐ App on Play Store
 - ☐ All workflows production-ready
 - ☐ **YOU LAUNCHED!** 🚀
-

UMESH'S MANTRA

"I automate the boring stuff."

Daily Affirmations:

- "n8n is my superpower"
- "I build mobile apps users love"
- "I ask Atharav when stuck"
- "I test on real devices"
- "I rest on Sundays"

Remember:

- n8n is unique skill (most developers don't know it!)
 - Your workflows save hours of manual work
 - Mobile users are your primary audience
 - Notifications must be perfect (users hate spam)
 - You're building something special! 💪
-

LET'S BUILD AMAZING MOBILE EXPERIENCE, UMESH! 🚀

You're going to automate everything with n8n - how cool is that?! 🤖

